

Programming Basics 1



10: Abstrakte Klassen & Interfaces

Sommersemester 2025

Simon Dewert

HAW Hamburg

Fakultät Design, Medien und Information

E52, Finkenau 35, 22081 Hamburg

simon.dewert@haw-hamburg.de

Übersicht dieser Lehreinheit

1. Abstrakte Klassen
2. Abstrakte Methoden
3. Schnittstellen (Interfaces)
4. Debugging in Eclipse

Abstrakte Klassen

Häufig dienen Basisklassen alleinig dazu, gemeinsame Attribute und Methoden der davon abgeleiteten Unterklassen zu **bündeln**, d. h. Gemeinsamkeiten zusammenfassend zu **abstrahieren**.

In diesem Fall ist es sinnvoll, diese Klasse(n) mit dem Schlüsselwort **abstract** zu kennzeichnen:

```
abstract class Basisklasse {  
  
}
```



UML 2.0 Notation
einer abstrakten Klasse

Von abstrakten Klassen können keine Instanzen gebildet (Objekte erzeugt) werden:

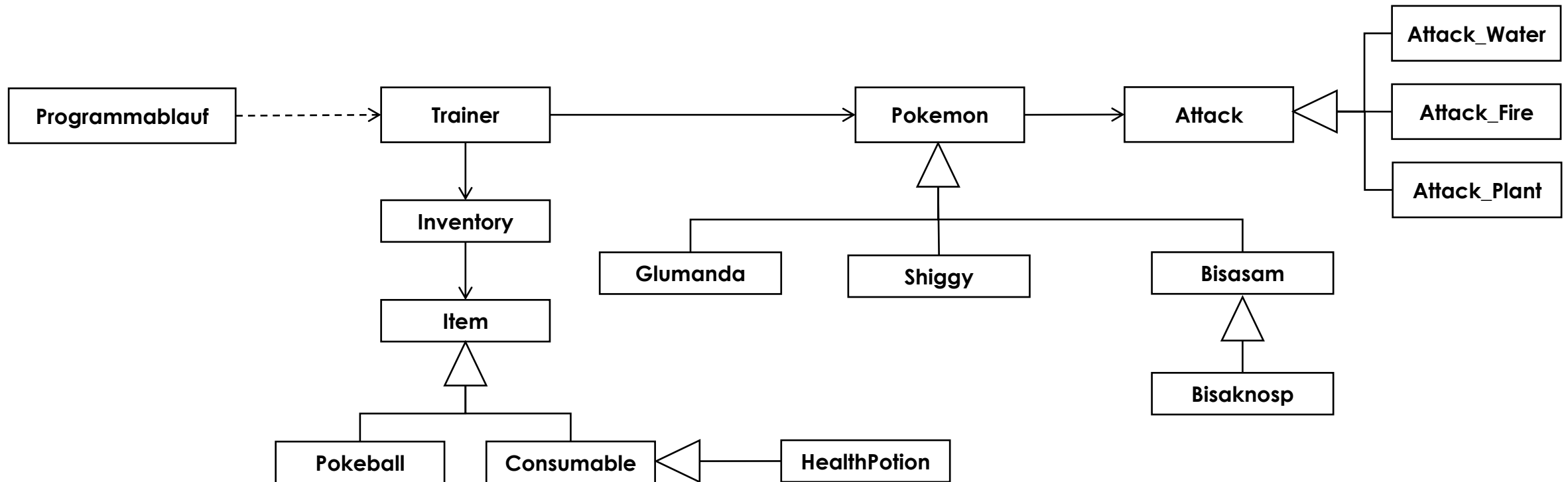
```
Basisklasse meinObjekt = new Basisklasse();
```

Aufgabe 10.1

5 min



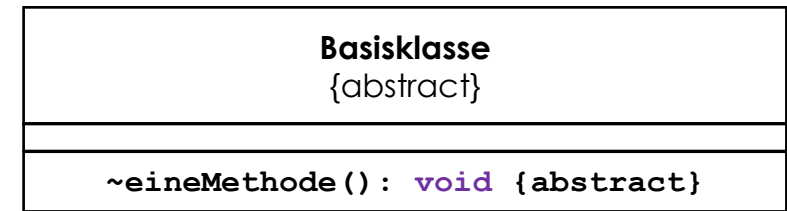
Prüfen Sie alle Klassen im **Pokemon** Projekt, ob diese in eine abstrakte Klasse umgewandelt werden können.



Abstrakte Methoden

Möchte man **in einer abstrakten Klasse erzwingen**, dass konkrete (= nicht-abstrakte) Unterklassen eine bestimmte Methode „**überschreiben**“, so muss diese Methode **auch** mit **abstract** gekennzeichnet werden und hat **keinen Rumpf**:

```
abstract class Basisklasse {  
    abstract void eineMethode(int einParameter);  
}
```



```
class AbgeleiteteKlasse extends Basisklasse {  
    void eineMethode(int Parameter) {  
        [...]  
    }  
}
```

Erzwungenes Überschreiben/Implementieren der Methode

Kein Rumpf: Direkter Abschluss mit Semikolon!!

Vorsicht: Abstrakte Methoden dürfen **NICHT** private sein

Abstrakte **Klassen** und **Methoden**

1. Abstrakte Klassen **können** abstrakte Methoden enthalten.
2. Eine Klasse, die abstrakte Methoden enthält **muss** selbst abstrakt sein.
3. Jede konkrete (nicht abstrakte) Klasse, die von einer abstrakten Klasse erbt, **muss** die abstrakten Methoden implementieren.

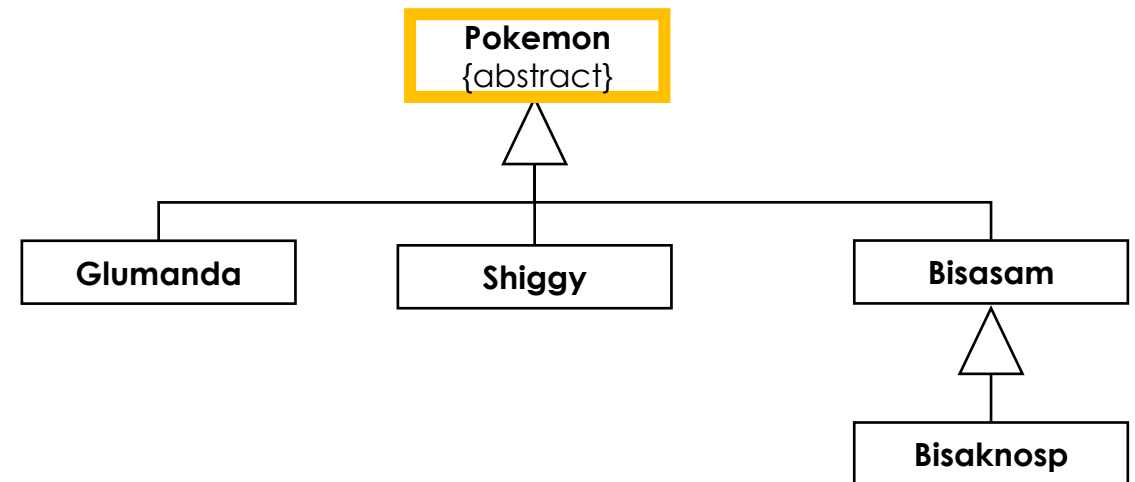
Aufgabe 10.2

15 min



Erzeugen in der Klasse **Pokemon** die **abstrakte** Methode `speak()` .

Alle speziellen Pokemonklassen (Glumanda, Shiggy, usw.) sollen diese Methode implementieren und zum Beispiel ihren Namen in die Konsole ausgeben.
(z.Bsp: „Shiggy!!“)



Immer wenn der Trainer das Pokemon ruft (**callPokemon()**), soll es sich mit seinem Namen oder einem netten Spruch melden (**speak()** aufrufen).

Schnittstellen (Interfaces)

Abstrakte Klassen, deren Attribute und Methoden **ausschließlich `public`** und deren Methoden **zusätzlich alle `abstract`** sind, können durch das Schlüsselwort **`interface`** auch als sogenannte **Schnittstelle** deklariert werden.

Dabei sind die Attribute und Methoden der Schnittstelle implizit, d.h. ohne weitere Angabe, **`public`** und die Methoden **immer `abstract`**:

```
abstract class AbstrakteKlasse {  
    public int wert = 0;  
  
    public abstract void methode1();  
    public abstract void methode2(int wert);  
}
```

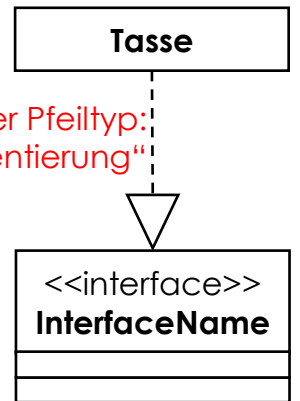
kann bzw. sollte

$\hat{=}$

```
interface AbstrakteKlasse {  
    public int wert = 0;  
  
    void methode1();  
    void methode2(int wert);  
}
```

Tasse

Neuer Pfeiltyp:
„Implementierung“



Implementierung einer Schnittstelle

Eine Klasse implementiert eine Schnittstelle durch Verwendung des Schlüsselworts **implements** im Klassenkopf:

```
class EineKlasse extends Basisklasse implements EineSchnittstelle {  
    Optional (nur bei zusätzlicher Vererbung)  
    public void methodeDerSchnittstelle()  
    {  
        [...]  
    }  
}
```

Zu überschreibende Methoden der Schnittstelle; diese **muss** public sein, da der Zugriffsmodifizierer beim Überschreiben einer Methode gleich oder großzügiger sein muss.

Schnittstellen können wie ein **Gütesiegel** aufgefasst werden:

Wenn eine Klasse eine Schnittstelle implementiert, kann fest davon ausgegangen werden, dass die Klasse die Methoden der Schnittstelle entweder **erbt oder selber formuliert**.

Schnittstellen in Java: Wichtige Regeln

- Schnittstellen haben keinen Konstruktor und können – wie abstrakte Klassen – **nicht instanziiert** werden.
- **Schnittstellen-Attribute** sind standardmäßig (also ohne explizite Angabe) **public**, **static** und **final**.
- **Schnittstellen-Methoden** sind standardmäßig **public**.
- Schnittstellen können über das Schlüsselwort **extends** **von anderen Schnittstellen abgeleitet** sein.
- Klassen **können mehrere Schnittstellen** implementieren.
(Separation durch Komma im Klassenkopf)

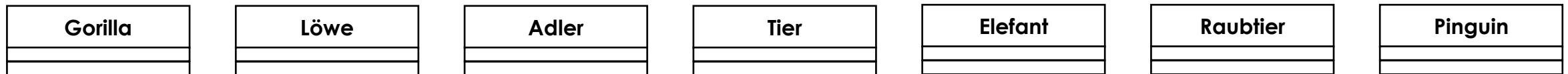
Aufgabe 10.3

20 min



Das wars mit Pokemon ☹️

Erstellen Sie ein neues Projekt mit folgenden Klassen.
Setzen Sie die Klassen mit einer **sinnvollen Hierarchie** in Beziehung (evtl. hilft ein UML-Klassendiagramm). Bestimmen sie dabei, welche Klassen **abstrakt** sein sollen:



Fügen Sie dann der Hierarchie eine **Schnittstelle** „**Schwimmfaehig**“ und „**Flugfaehig**“ hinzu und sorgen Sie dafür, dass alle geeigneten Klassen diese Schnittstelle implementieren!

Bitte die folgenden Methoden und Attribute bestmöglich zuordnen:

Methoden: **gibLaut()**, **fliegen()**, **jagen()**, **schwimmen()**, **toString()**

Attribute: **name**, **alter**, **gewicht**

Debugging

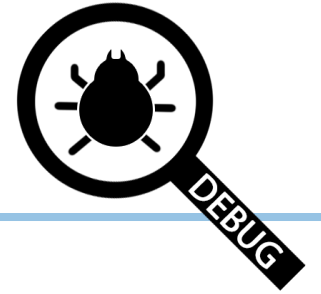


Debugging ist ein entscheidender Prozess bei der Softwareentwicklung, der es Entwicklern ermöglicht, **Fehler** in ihrem Code zu **finden** und zu **beheben**

Eclipse bietet leistungsstarke Debugging-Funktionen, um Entwicklern bei der Fehlerbehebung in ihrem Code zu helfen.

Durch den Debug-Modus, **Breakpoints** und spezielle Ansichten können Entwickler den Code schrittweise ausführen, Variablen überwachen und Fehler effizient identifizieren und beheben.

Breakpoint



Doppel
klick

```
6 public class DebuggingBeispiel {
7     public static void main(String[] args) {
8
9         Random rnd = new Random();
10
11         int a = 1 + rnd.nextInt(97);
12         int b = a + 2 + rnd.nextInt(99-a);
13
14         getNumberBetween(a, b);
15     }
16 }
```

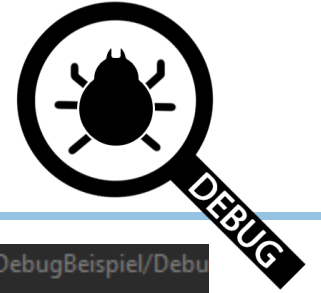
```
6 public class DebuggingBeispiel {
7     public static void main(String[] args) {
8
9         Random rnd = new Random();
10
11         int a = 1 + rnd.nextInt(97);
12         int b = a + 2 + rnd.nextInt(99-a);
13
14         getNumberBetween(a, b);
15     }
16 }
```

Ein Breakpoint ist ein Marker, den ein Entwickler in seinem Code setzen kann, um die Ausführung an einer bestimmten Stelle zu stoppen.

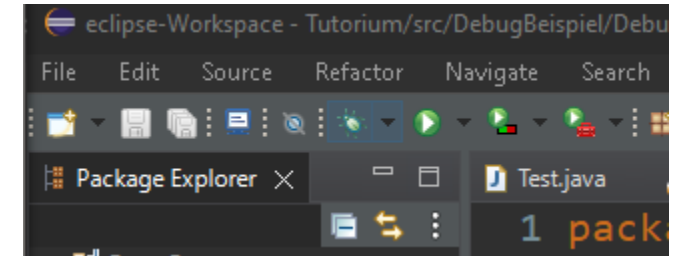
Wenn der Debugger auf einen Breakpoint trifft, wird die Ausführung des Programms angehalten, und der Entwickler kann den aktuellen Zustand des Codes überprüfen, Variablenwerte analysieren und die weiteren Schritte steuern.

Breakpoint erzeugen/entfernen: Doppelklicke auf die Zeilenzahl.

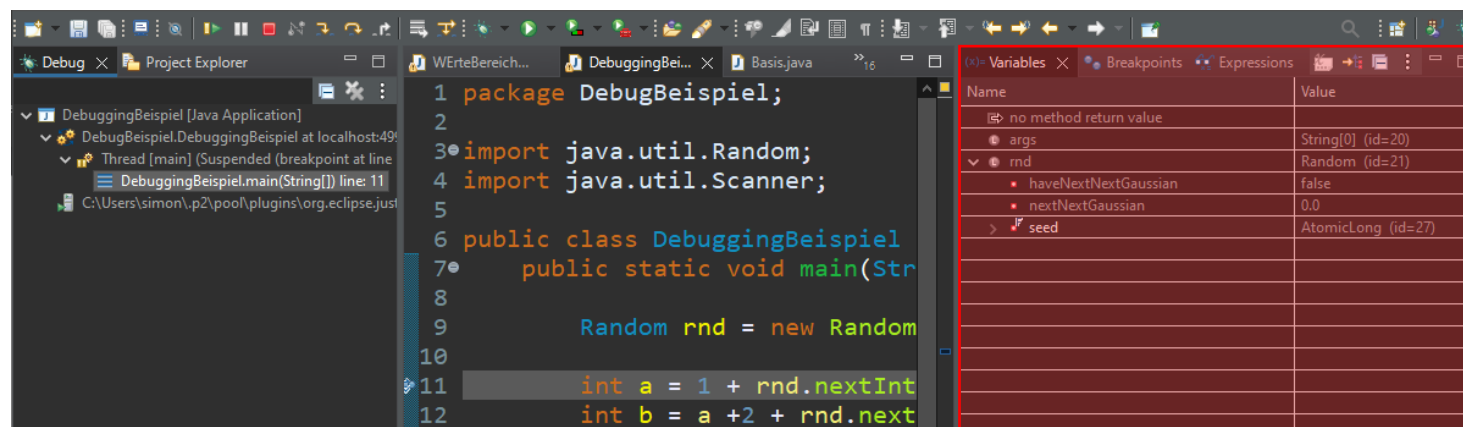
Debug Modus



Das kleine Käfer-Symbol neben dem Play-Button, startet das Programm im Debug-Modus.



Sobald das Programm jetzt an eine Stelle mit Breakpoint trifft, wird der Programmfluss angehalten und der aktuelle Inhalt jeder verfügbaren Variable kann angesehen werden.

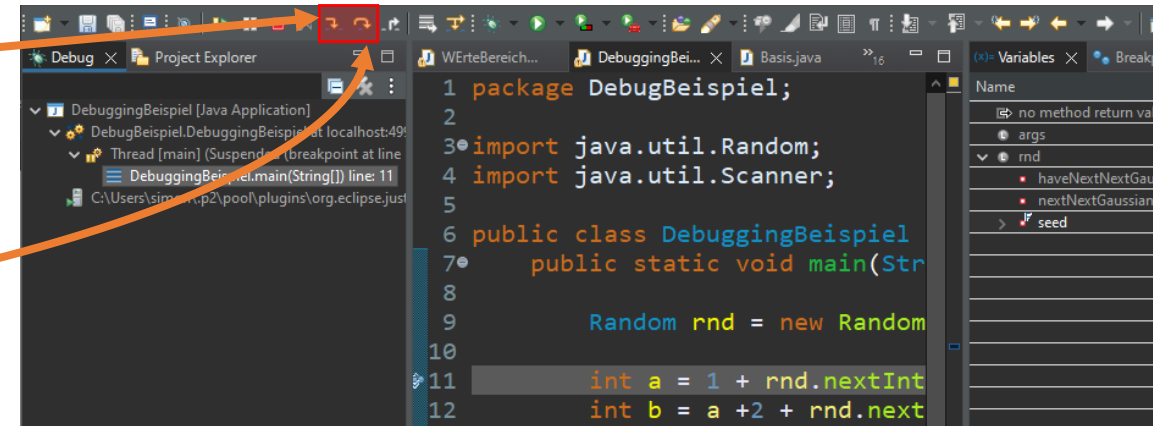


Debug Modus



Schritt für Schritt durch den Code:

- **Step Into (F7)** — Steigt in jeden Funktionsaufruf und zeigt alle Schritte im Inneren.
- **Step Over (F8)** — Bleibt in der aktuellen Funktion und springt in die nächste Zeile.
(Funktion wird im Hintergrund durchlaufen)



Viele Dank für Ihre Aufmerksamkeit



Hausaufgabe 10



Probiert den Debugging-Mode im Pokemon Projekt aus.